

Task 1

1. Espacio de estados S

La acción anterior no importa, lo que importa es dónde está el dron ahora, porque podemos definir estados por celda dentro de la "ciudad", como respuesta general pero con otros parámetros para darle contexto.

Pero para los drones no alcanza usar solo una celda. Por ejemplo, si el dron está en (3, 4), no puede decidir bien si no sabe si todavía lleva el paquete, si ya lo entregó o cuánta batería tiene. Dos drones en la misma casilla podrían tener que hacer cosas totalmente distintas.

Entonces proponemos este estado:

$$s = (x, y, b, p, d)$$

- (x, y) : posición del dron en la cuadrícula. Se necesita para saber qué movimientos son posibles y cuánto falta para llegar.
- b : batería restante, por ejemplo de 0 a 10. Se necesita porque determina si el dron puede completar el viaje y volver a la base.
- p : estado del paquete: 1 si el dron todavía lo lleva y 0 si ya lo entregó. Antes de entregar debe ir al destino; después debe regresar.
- d : coordenada del destino asignado. Es necesaria si los pedidos pueden ir a distintas casillas.

La base es siempre la casilla central (2,2), por eso no hace falta guardarla dentro de todos los estados. También suponemos que los edificios o zonas prohibidas son fijos y conocidos: pueden estar definidos una sola vez en el mapa. Omitimos cosas como el nombre del cliente, el color del dron o el número del pedido porque no cambian el resultado del siguiente movimiento.

Estamos omitiendo viento, tráfico de otros drones y estado real de la batería. Es una simplificación. La consecuencia es que el MDP no representa toda la realidad: dos estados que se ven iguales podrían tener riesgos distintos. Aun así, sirve como primer modelo para saber si la idea funciona antes de hacer algo mucho más complejo.

La propiedad de Markov se cumple razonablemente porque, al conocer posición, batería, paquete y destino, podemos decidir la siguiente acción y estimar qué pasará sin necesitar la acción anterior.

2. Espacio de acciones A

Las acciones son: **norte** , **sur** , **este** , **oeste** , **entregar** y **esperar** . Son acciones discretas porque el dron escoge una opción de una lista; no puede escoger cualquier dirección decimal, como 37.4 grados.

Hay acciones que se deben restringir:

- No puede ir al norte si ya está en la fila superior, ni salir por ningún borde del tablero.
- No puede ir hacia una casilla con edificio o zona prohibida.
- Solo puede usar **entregar** si está exactamente en el destino y aún tiene el paquete.

Esto se modela con $A(s)$, que significa "las acciones permitidas en el estado s ". Por ejemplo, en una esquina solo habrá dos movimientos válidos. También podríamos dejar que el dron elija una acción inválida, pero entonces lo dejamos en la misma casilla y le damos un castigo; para el laboratorio es más claro simplemente no incluirla en $A(s)$.

3. Función de recompensa $r(s,a,s')$

La recompensa es el puntaje que guía al dron. La empresa quiere entregar rápido, gastar poca batería y no tener accidentes. Como estas metas compiten entre sí, usamos estas recompensas:

- Cada movimiento válido: **-1** . Así evita dar vueltas y hace entregas eficientes.
- Cada unidad de batería usada: **-0.5** . Así prefiere rutas que consuman menos.
- Entregar el paquete: **+30** .
- Volver a la base después de entregar: **+40** .
- Chocar, entrar a una zona prohibida o quedarse sin batería: **-100** y termina la misión.

El castigo de seguridad es el más grande porque un accidente cuesta mucho más que tardar un paso extra. Si el premio por entregar fuera demasiado alto, el agente podría tomar rutas peligrosas. Si el costo de batería fuera demasiado alto, podría evitar hacer entregas largas aunque sean necesarias. Si el castigo por accidente fuera pequeño, podría aprender a arriesgarse demasiado.

4. Función de transición $p(s' | s,a)$

En este ejemplo de tablero básico las transiciones podrían ser deterministas: si voy al este, siempre avanzo una casilla. Para una ciudad real es mejor usar una transición estocástica, o sea, con probabilidades, porque hay viento, errores de GPS, obstáculos temporales o fallos de navegación.

Ejemplos:

1. Si el dron está en $(1,1,7,1,d)$ y usa **este** , tiene probabilidad **0.90** de llegar a $(2,1,6,1,d)$ y probabilidad **0.10** de quedarse en $(1,1,6,1,d)$ por una ráfaga de

viento. Casi siempre el vuelo funciona, pero aun cuando no avanza gasta batería tratando de hacerlo, y sería de considerar el viento para el gasto de batería.

2. Si está en $(3, 3, 5, 1, d)$ y usa **norte**, tiene probabilidad 0.85 de llegar a $(3, 2, 4, 1, d)$, probabilidad 0.10 de quedarse donde está y probabilidad 0.05 de caer o tener un accidente. La probabilidad pequeña de accidente representa una condición urbana peligrosa.
3. Si está en el destino $(4, 4, 4, 1, (4, 4))$ y usa **entregar**, cambia a $(4, 4, 4, 0, (4, 4))$ con probabilidad 0.99 ; con probabilidad 0.01 la entrega no se confirma y permanece con el paquete. Es normal que entregar sea muy confiable, pero no perfecto, puede haber alguien que no reciba, o incluso problemas con el paquete en el momento de entrega.

Estos números son supuestos iniciales. Una empresa real debería estimarlos con datos de vuelos y accidentes.

5. Factor de descuento gamma

Proponemos $\gamma = 0.95$. Gamma. En este caso queremos que la entrega y el regreso sean importantes, pero que el dron también prefiera terminarlos pronto. Un valor de 0.95 hace que una ruta segura un poco más larga siga valiendo la pena, sin hacer que el agente pierda demasiado tiempo dando vueltas.

Task 2

1. Casos donde se viola Markov

Caso 1: el viento. Dos drones pueden estar en la misma posición, con la misma batería y paquete, pero uno puede tener viento fuerte y el otro viento normal. Si no guardamos el viento, la probabilidad de avanzar o fallar no es igual en ambos casos. Para restaurar Markov agregamos una variable w al estado: $w = \text{normal, suave o fuerte}$. El costo es que el número de estados se multiplica por tres. Si además guardamos dirección y velocidad del viento, el modelo crece aún más.

Caso 2: batería desgastada. Dos drones con batería nivel 5 no necesariamente gastan igual: uno puede tener una batería nueva y otro una dañada. Sin incluir esa condición, el siguiente estado depende de la historia de uso de la batería. Podemos agregar h , salud de batería: $\text{buena, media o mala}$. El costo vuelve a multiplicar los estados y necesitamos estimar esa salud con sensores.

También podría pasar con otros drones: para saber el riesgo de choque habría que agregar las posiciones y tal vez direcciones de los drones cercanos. Eso haría crecer el espacio de estados muy rápido, porque cada dron adicional puede estar en muchas casillas.

2. ¿El dron observa todo el estado?

No es muy razonable asumir que ve el estado completo. El GPS puede tener error; el dron no conoce perfectamente el viento entre edificios; la salud real de la batería es una estimación; y puede no detectar a tiempo aves, cables, obstáculos o drones que estén ocultos.

Un MDP supone que el agente conoce el estado real, por ejemplo que sabe con certeza que está en $(2, 3)$ y que el viento es suave. Eso hace que el modelo sea más fácil de programar y es aceptable para este laboratorio. En la realidad sería más apropiado un POMDP, que es un MDP parcialmente observable. En un POMDP el dron no ve todo: recibe observaciones imperfectas de GPS, cámara y sensores, y decide usando una estimación de la situación. Es más realista, pero más difícil de resolver.

3. ¿Tarea episódica o continua?

Para este problema la modelaría como episódica. Un episodio inicia cuando el dron sale de la base con un paquete y termina cuando lo entrega y regresa a la base, o cuando ocurre un accidente o se queda sin batería. Cada pedido es una misión completa y separada.

Con una tarea episódica tiene sentido dar premios grandes al entregar y al regresar, y usar $\gamma = 0.95$: queremos completar esta misión pronto y de manera segura.

Si se modelara como continua, el dron nunca termina: al volver toma otro paquete y sigue trabajando. Entonces la recompensa debería incluir beneficios repetidos por entregas y costos constantes por tiempo, batería y riesgo; no dependería solo de un premio final. También convendría un gamma más cercano a 1, por ejemplo 0.99 , porque importan mucho las entregas futuras de toda la jornada.

Task 3 (Entrega Final)

Objetivo y alcance

Se implementa desde cero un MDP para una ciudad de drones de 5×5 y un evaluador iterativo de políticas. No se utiliza Gymnasium ni ninguna librería de aprendizaje por refuerzo.

Para mantener consistencia con la Tarea 1, el estado conserva la posición y la fase del paquete:

$s = (\text{fila}, \text{columna}, p)$, donde $p = 1$ significa que el dron lleva el paquete y $p = 0$ que ya lo entregó y debe regresar.

El destino se fija en $(0, 4)$ y la base en $(2, 2)$, por lo que no es necesario repetir **d** en cada estado. La batería se considera suficiente para una misión y su consumo se incorpora en la recompensa de cada vuelo (-1 por tiempo y -0.5 por batería). Esta abstracción permite representar los valores de todos los estados espaciales en grids legibles, sin perder las dos fases de entrega y retorno.

El episodio termina cuando el dron vuelve a la base después de entregar. El estado terminal se representa de forma separada y tiene valor cero.

1. Parámetros del experimento

El criterio de convergencia es configurable mediante **THETA**. En cada iteración se aplica de manera síncrona el operador de Bellman de expectativa y se termina cuando el cambio máximo entre dos tablas consecutivas es menor que ese umbral.

```
In [1]: from collections import OrderedDict

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from IPython.display import Markdown, display

GRID_SIZE = 5
BASE = (2, 2)
DESTINATION = (0, 4)

GAMMA = 0.95
THETA = 1e-8
MAX_ITERATIONS = 20_000

MOVE_SUCCESS_PROBABILITY = 0.90
MOVE_REWARD = -1.5          # -1 por tiempo y -0.5 por batería
WAIT_REWARD = -1.0
DELIVERY_REWARD = 30.0
RETURN_REWARD = 40.0
DELIVERY_SUCCESS_PROBABILITY = 0.99

TERMINAL = "terminal"

print(f"Grid: {GRID_SIZE}x{GRID_SIZE} | base={BASE} | destino={DESTINATION}")
print(f"gamma={GAMMA} | theta={THETA}")
```

```
Grid: 5x5 | base=(2, 2) | destino=(0, 4)
gamma=0.95 | theta=1e-08
```

2. MDP completo implementado desde cero

- **Estados:** las 25 posiciones con paquete, las 24 posiciones de retorno (la base ya es terminal en esa fase) y un estado terminal.

- **Acciones:** norte, sur, este, oeste, esperar y entregar. ENTREGAR solo está disponible en el destino mientras se lleva el paquete.
- **Transición de vuelo:** con probabilidad 0.90 el movimiento ocurre y con 0.10 el viento deja al dron en la misma celda. Las acciones que salen del grid no pertenecen a $A(s)$.
- **Entrega:** funciona con probabilidad 0.99; si falla, el dron conserva el paquete.
- **Recompensas:** cada vuelo cuesta -1.5, esperar cuesta -1, entregar da +30 y volver a la base después de entregar da +40.

El diccionario $P[s][a]$ contiene tuplas (probabilidad, siguiente_estado, recompensa, terminado), por lo que especifica conjuntamente la función de transición y la función de recompensa.

```
In [2]: class DroneCityMDP:
        # MDP tabular de una misión de entrega y retorno en un GridWorld 5x5.

        MOVEMENT_DELTAS = OrderedDict(
            {
                "NORTE": (-1, 0),
                "SUR": (1, 0),
                "ESTE": (0, 1),
                "OESTE": (0, -1),
            }
        )

        def __init__(
            self,
            grid_size=GRID_SIZE,
            base=BASE,
            destination=DESTINATION,
            move_success_probability=MOVE_SUCCESS_PROBABILITY,
            delivery_success_probability=DELIVERY_SUCCESS_PROBABILITY,
        ):
            self.grid_size = grid_size
            self.base = base
            self.destination = destination
            self.move_success_probability = move_success_probability
            self.delivery_success_probability = delivery_success_probability

            carrying_states = [
                (row, col, 1)
                for row in range(grid_size)
                for col in range(grid_size)
            ]
            returning_states = [
                (row, col, 0)
                for row in range(grid_size)
                for col in range(grid_size)
                if (row, col) != base
            ]

            self.nonterminal_states = carrying_states + returning_states
```

```

self.states = self.nonterminal_states + [TERMINAL]
self.actions = {
    state: self.available_actions(state) for state in self.states
}
self.P = {
    state: {
        action: self.transition_outcomes(state, action)
        for action in self.actions[state]
    }
    for state in self.states
    if state != TERMINAL
}

def inside_grid(self, row, col):
    return 0 <= row < self.grid_size and 0 <= col < self.grid_size

def available_actions(self, state):
    if state == TERMINAL:
        return tuple()

    row, col, package = state
    valid_actions = []
    for action, (delta_row, delta_col) in self.MOVEMENT_DELTAS.items():
        if self.inside_grid(row + delta_row, col + delta_col):
            valid_actions.append(action)

    valid_actions.append("ESPERAR")
    if package == 1 and (row, col) == self.destination:
        valid_actions.append("ENTREGAR")
    return tuple(valid_actions)

def transition_outcomes(self, state, action):
    if action not in self.actions[state]:
        raise ValueError(f"Acción {action!r} no válida en el estado {state!r}.")

    if action == "ESPERAR":
        return [(1.0, state, WAIT_REWARD, False)]

    if action == "ENTREGAR":
        delivered_state = (*self.destination, 0)
        failure_probability = 1.0 - self.delivery_success_probability
        return [
            (
                self.delivery_success_probability,
                delivered_state,
                DELIVERY_REWARD,
                False,
            ),
            (failure_probability, state, WAIT_REWARD, False),
        ]

    row, col, package = state
    delta_row, delta_col = self.MOVEMENT_DELTAS[action]
    intended_position = (row + delta_row, col + delta_col)

    returned_to_base = package == 0 and intended_position == self.base

```

```

        if returned_to_base:
            success_next_state = TERMINAL
            success_reward = MOVE_REWARD + RETURN_REWARD
            success_done = True
        else:
            success_next_state = (*intended_position, package)
            success_reward = MOVE_REWARD
            success_done = False

    failure_probability = 1.0 - self.move_success_probability
    return [
        (
            self.move_success_probability,
            success_next_state,
            success_reward,
            success_done,
        ),
        (failure_probability, state, MOVE_REWARD, False),
    ]

mdp = DroneCityMDP()

# Verificaciones estructurales del MDP.
assert len(mdp.states) == 50 # 25 con paquete + 24 de retorno + terminal
for state, action_transitions in mdp.P.items():
    for action, outcomes in action_transitions.items():
        probability_sum = sum(probability for probability, _ in outcomes)
        assert np.isclose(probability_sum, 1.0), (state, action, probability_sum)
        assert all(next_state in mdp.states for _, next_state, _, _ in outcomes)

print(f"Estados totales: {len(mdp.states)}")
print(f"Estados no terminales: {len(mdp.nonterminal_states)}")
print("Todas las distribuciones de transición suman 1.0.")

```

Estados totales: 50

Estados no terminales: 49

Todas las distribuciones de transición suman 1.0.

3. Tres políticas

1. **Uniforme aleatoria:** reparte la probabilidad por igual entre todas las acciones válidas.
2. **Determinista fija:** durante la entrega sigue un recorrido serpenteante predefinido que eventualmente visita el destino; durante el retorno se dirige a la base. No utiliza valores ni aprende.
3. **Greedy mejorada:** se obtiene aplicando una mejora de política de un paso sobre los valores de la política determinista anterior.

```

In [3]: def one_hot_policy(valid_actions, selected_action):
        return {
            action: float(action == selected_action)
            for action in valid_actions
        }

```



```

def create_uniform_policy(mdp):
    policy = {}
    for state in mdp.nonterminal_states:
        probability = 1.0 / len(mdp.actions[state])
        policy[state] = {
            action: probability for action in mdp.actions[state]
        }
    return policy

def action_between(current_position, next_position):
    delta = (
        next_position[0] - current_position[0],
        next_position[1] - current_position[1],
    )
    for action, action_delta in DroneCityMDP.MOVEMENT_DELTAS.items():
        if action_delta == delta:
            return action
    raise ValueError(f"Las posiciones {current_position} y {next_position} no son a")

def create_fixed_deterministic_policy(mdp):
    # Camino serpenteante que cubre el grid y termina en el destino (0, 4).
    snake_path = []
    for row in range(mdp.grid_size - 1, -1, -1):
        columns = (
            range(mdp.grid_size)
            if row % 2 == 0
            else range(mdp.grid_size - 1, -1, -1)
        )
        snake_path.extend((row, col) for col in columns)

    assert snake_path[-1] == mdp.destination
    next_on_snake = {
        snake_path[index]: snake_path[index + 1]
        for index in range(len(snake_path) - 1)
    }

    policy = {}
    for state in mdp.nonterminal_states:
        row, col, package = state
        position = (row, col)

        if package == 1:
            if position == mdp.destination:
                selected_action = "ENTREGAR"
            else:
                selected_action = action_between(position, next_on_snake[position])
        else:
            base_row, base_col = mdp.base
            if col < base_col:
                selected_action = "ESTE"
            elif col > base_col:
                selected_action = "OESTE"

```

```

        elif row < base_row:
            selected_action = "SUR"
        else:
            selected_action = "NORTE"

    policy[state] = one_hot_policy(
        mdp.actions[state], selected_action
    )
return policy

uniform_policy = create_uniform_policy(mdp)
fixed_policy = create_fixed_deterministic_policy(mdp)

for policy in (uniform_policy, fixed_policy):
    for state, probabilities in policy.items():
        assert set(probabilities) == set(mdp.actions[state])
        assert np.isclose(sum(probabilities.values()), 1.0)

print("Políticas uniforme y determinista construidas y validadas.")

```

Políticas uniforme y determinista construidas y validadas.

4. Evaluación iterativa mediante Bellman

Para una política fija π , cada actualización síncrona calcula:

$$V_{k+1}(s) = \sum_a \pi(a|s) \sum_{s'} p(s'|s,a) [r(s,a,s') + \gamma V_k(s')]$$

El valor terminal se mantiene en cero. La función devuelve la tabla final, el número de iteraciones y el historial del cambio máximo Δ .

```

In [4]: def evaluate_policy_iteratively(
    mdp,
    policy,
    gamma=GAMMA,
    theta=THETA,
    max_iterations=MAX_ITERATIONS,
):
    if not (0 <= gamma < 1):
        raise ValueError("gamma debe estar en el intervalo [0, 1).")
    if theta <= 0:
        raise ValueError("theta debe ser mayor que cero.")

    values = {state: 0.0 for state in mdp.states}
    delta_history = []

    for iteration in range(1, max_iterations + 1):
        new_values = values.copy()
        maximum_change = 0.0

        for state in mdp.nonterminal_states:
            updated_value = 0.0
            for action, action_probability in policy[state].items():

```

```

        if action_probability == 0:
            continue
        for (
            transition_probability,
            next_state,
            reward,
            done,
        ) in mdp.P[state][action]:
            continuation = 0.0 if done else gamma * values[next_state]
            updated_value += (
                action_probability
                * transition_probability
                * (reward + continuation)
            )

        new_values[state] = updated_value
        maximum_change = max(
            maximum_change, abs(updated_value - values[state])
        )

        values = new_values
        delta_history.append(maximum_change)
        if maximum_change < theta:
            return values, iteration, delta_history

    raise RuntimeError(
        f"La evaluación no convergió después de {max_iterations} iteraciones."
    )

def action_value(mdp, state, action, values, gamma=GAMMA):
    return sum(
        transition_probability
        * (reward + (0.0 if done else gamma * values[next_state]))
        for transition_probability, next_state, reward, done
        in mdp.P[state][action]
    )

def greedy_policy_from_values(mdp, values, gamma=GAMMA):
    greedy_policy = {}
    for state in mdp.nonterminal_states:
        q_values = {
            action: action_value(mdp, state, action, values, gamma)
            for action in mdp.actions[state]
        }
        best_value = max(q_values.values())
        # Desempate determinista según el orden declarado de las acciones.
        best_action = next(
            action
            for action in mdp.actions[state]
            if np.isclose(q_values[action], best_value)
        )
        greedy_policy[state] = one_hot_policy(
            mdp.actions[state], best_action
        )

```

```

return greedy_policy

uniform_values, uniform_iterations, uniform_deltas = (
    evaluate_policy_iteratively(mdp, uniform_policy)
)
fixed_values, fixed_iterations, fixed_deltas = (
    evaluate_policy_iteratively(mdp, fixed_policy)
)

# La tercera política se deriva explícitamente de V de la política anterior.
greedy_policy = greedy_policy_from_values(mdp, fixed_values)
greedy_values, greedy_iterations, greedy_deltas = (
    evaluate_policy_iteratively(mdp, greedy_policy)
)

results = OrderedDict(
    {
        "Uniforme aleatoria": {
            "policy": uniform_policy,
            "values": uniform_values,
            "iterations": uniform_iterations,
            "deltas": uniform_deltas,
        },
        "Determinista fija": {
            "policy": fixed_policy,
            "values": fixed_values,
            "iterations": fixed_iterations,
            "deltas": fixed_deltas,
        },
        "Greedy sobre V determinista": {
            "policy": greedy_policy,
            "values": greedy_values,
            "iterations": greedy_iterations,
            "deltas": greedy_deltas,
        },
    }
)

print("Las tres políticas convergieron con el mismo criterio theta.")

```

Las tres políticas convergieron con el mismo criterio theta.

5. Tablas de valores y comparación

Cada política produce dos tablas 5×5 : la primera contiene todos los estados donde el dron aún lleva el paquete y la segunda los estados de retorno. En la tabla de retorno, la base aparece como NaN porque entrar en ella lleva directamente al estado terminal; el valor del estado terminal es 0.

```

In [5]: def value_grid(values, package):
        grid = np.full((mdp.grid_size, mdp.grid_size), np.nan)
        for row in range(mdp.grid_size):
            for col in range(mdp.grid_size):

```

```

        state = (row, col, package)
        if state in values:
            grid[row, col] = values[state]
    return pd.DataFrame(
        grid,
        index=[f"fila {row}" for row in range(mdp.grid_size)],
        columns=[f"col {col}" for col in range(mdp.grid_size)],
    )

initial_state = (*BASE, 1)
summary = pd.DataFrame(
    [
        {
            "Política": policy_name,
            "Iteraciones": result["iterations"],
            "Delta final": result["deltas"][-1],
            "V(base, lleva paquete)": result["values"][initial_state],
        }
        for policy_name, result in results.items()
    ]
).set_index("Política")

display(summary.style.format(
    {
        "Delta final": "{:.3e}",
        "V(base, lleva paquete)": "{:.4f}",
    }
))

for policy_name, result in results.items():
    display(Markdown(f"### {policy_name} – {result['iterations']} iteraciones"))
    display(Markdown("**Fase de entrega (p = 1)**"))
    display(value_grid(result["values"], package=1).round(3))
    display(Markdown("**Fase de retorno (p = 0)**"))
    display(value_grid(result["values"], package=0).round(3))

print(f"V(terminal) = {uniform_values[TERMINAL]:.1f} para todas las políticas.")

```

	Iteraciones	Delta final	V(base, lleva paquete)
Política			
Uniforme aleatoria	337	9.645e-09	-24.2360
Determinista fija	50	3.729e-09	13.4983
Greedy sobre V determinista	29	3.521e-09	38.5382

Uniforme aleatoria — 337 iteraciones

Fase de entrega (p = 1)

	col 0	col 1	col 2	col 3	col 4
fila 0	-24.515	-23.660	-21.161	-15.565	-2.970
fila 1	-24.992	-24.406	-22.776	-19.772	-15.565
fila 2	-25.467	-25.146	-24.236	-22.776	-21.161
fila 3	-25.789	-25.640	-25.146	-24.406	-23.660
fila 4	-25.860	-25.789	-25.467	-24.992	-24.515

Fase de retorno (p = 0)

	col 0	col 1	col 2	col 3	col 4
fila 0	-8.018	-6.382	-4.090	-6.382	-8.018
fila 1	-6.382	-2.099	5.971	-2.099	-6.382
fila 2	-4.090	5.971	NaN	5.971	-4.090
fila 3	-6.382	-2.099	5.971	-2.099	-6.382
fila 4	-8.018	-6.382	-4.090	-6.382	-8.018

Determinista fija — 50 iteraciones

Fase de entrega (p = 1)

	col 0	col 1	col 2	col 3	col 4
fila 0	38.538	42.546	46.789	51.279	56.032
fila 1	34.752	31.174	27.794	24.601	21.585
fila 2	8.825	11.095	13.498	16.042	18.735
fila 3	6.680	4.653	2.739	0.930	-0.779
fila 4	-8.007	-6.721	-5.360	-3.919	-2.393

Fase de retorno (p = 0)

	col 0	col 1	col 2	col 3	col 4
fila 0	27.443	30.802	34.358	30.802	27.443
fila 1	30.802	34.358	38.122	34.358	30.802
fila 2	34.358	38.122	NaN	38.122	34.358
fila 3	30.802	34.358	38.122	34.358	30.802
fila 4	27.443	30.802	34.358	30.802	27.443

Greedy sobre V determinista — 29 iteraciones

Fase de entrega ($p = 1$)

	col 0	col 1	col 2	col 3	col 4
fila 0	38.538	42.546	46.789	51.279	56.032
fila 1	34.752	38.538	42.546	46.789	51.279
fila 2	31.174	34.752	38.538	42.546	46.789
fila 3	27.794	31.174	34.752	38.538	42.546
fila 4	24.601	27.794	31.174	34.752	38.538

Fase de retorno ($p = 0$)

	col 0	col 1	col 2	col 3	col 4
fila 0	27.443	30.802	34.358	30.802	27.443
fila 1	30.802	34.358	38.122	34.358	30.802
fila 2	34.358	38.122	NaN	38.122	34.358
fila 3	30.802	34.358	38.122	34.358	30.802
fila 4	27.443	30.802	34.358	30.802	27.443

$V(\text{terminal}) = 0.0$ para todas las políticas.

6. Visualización del GridWorld

Los seis mapas usan la misma escala de color para que la comparación sea directa. En cada celda se muestra el valor y se superpone la acción greedy respecto de la tabla evaluada: , , , ,  para esperar y  para entregar.

```
In [6]: ACTION_SYMBOLS = {
    "NORTE": "↑",
    "SUR": "↓",
    "ESTE": "→",
    "OESTE": "←",
    "ESPERAR": "•",
    "ENTREGAR": "✓",
}

def selected_action(policy, state):
    return max(policy[state], key=policy[state].get)

# Para cada V mostramos la política greedy que inducen esos valores.
greedy_overlays = {
    policy_name: greedy_policy_from_values(mdp, result["values"])
    for policy_name, result in results.items()
}
```

```

all_nonterminal_values = [
    result["values"][state]
    for result in results.values()
    for state in mdp.nonterminal_states
]
color_min, color_max = min(all_nonterminal_values), max(all_nonterminal_values)

fig, axes = plt.subplots(
    2,
    len(results),
    figsize=(18, 10),
    constrained_layout=True,
)

for column, (policy_name, result) in enumerate(results.items()):
    overlay_policy = greedy_overlays[policy_name]
    for row_index, package in enumerate((1, 0)):
        axis = axes[row_index, column]
        grid = value_grid(result["values"], package).to_numpy()
        masked_grid = np.ma.masked_invalid(grid)
        image = axis.imshow(
            masked_grid,
            cmap="RdYlGn",
            vmin=color_min,
            vmax=color_max,
        )

        for row in range(mdp.grid_size):
            for col in range(mdp.grid_size):
                state = (row, col, package)
                if state not in mdp.nonterminal_states:
                    axis.text(
                        col,
                        row,
                        "BASE\nterminal",
                        ha="center",
                        va="center",
                        fontsize=10,
                        fontweight="bold",
                    )
                    continue

                action = selected_action(overlay_policy, state)
                axis.text(
                    col,
                    row,
                    f"{result['values'][state]:.1f}\n{ACTION_SYMBOLS[action]}",
                    ha="center",
                    va="center",
                    fontsize=9,
                    fontweight="semibold",
                )

        # Marcas de contexto de la ciudad.
        axis.scatter(

```



```

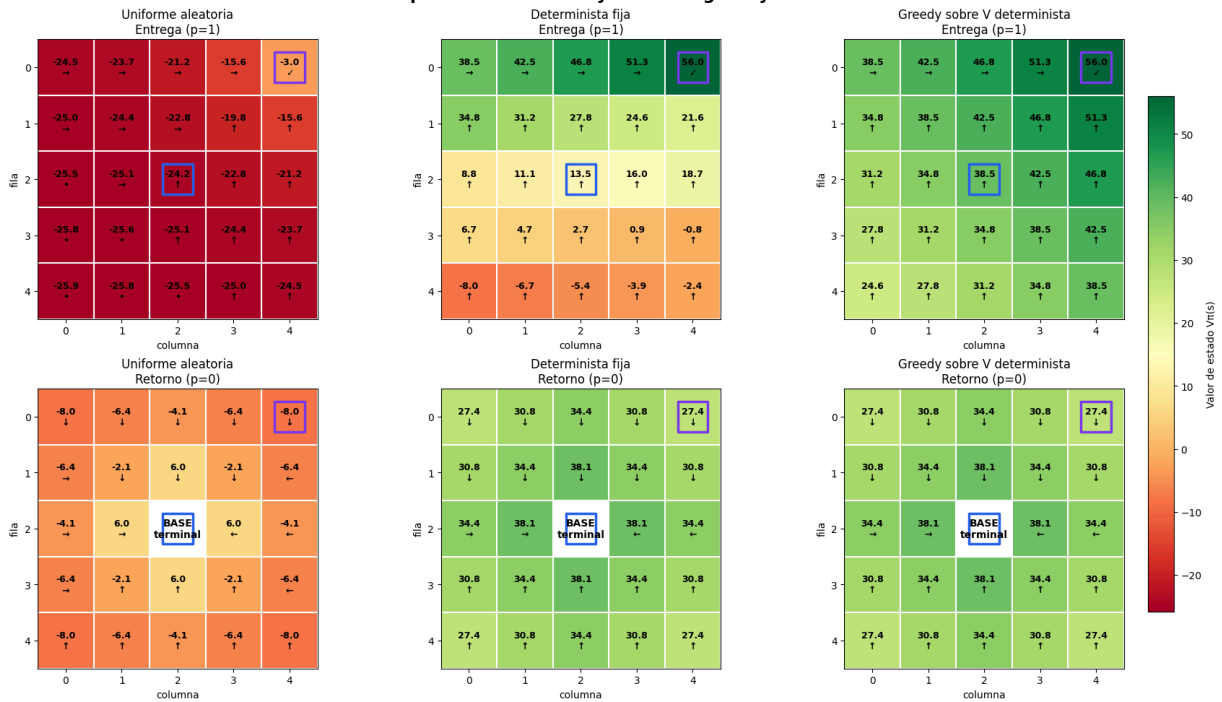
        [mdp.base[1]],
        [mdp.base[0]],
        marker="s",
        s=900,
        facecolors="none",
        edgecolors="#2563eb",
        linewidths=2.5,
    )
    axis.scatter(
        [mdp.destination[1]],
        [mdp.destination[0]],
        marker="s",
        s=900,
        facecolors="none",
        edgecolors="#7c3aed",
        linewidths=2.5,
    )

    phase_name = "Entrega (p=1)" if package == 1 else "Retorno (p=0)"
    axis.set_title(f"{policy_name}\n{phase_name}", fontsize=12)
    axis.set_xticks(range(mdp.grid_size), labels=range(mdp.grid_size))
    axis.set_yticks(range(mdp.grid_size), labels=range(mdp.grid_size))
    axis.set_xlabel("columna")
    axis.set_ylabel("fila")
    axis.set_xticks(np.arange(-0.5, mdp.grid_size, 1), minor=True)
    axis.set_yticks(np.arange(-0.5, mdp.grid_size, 1), minor=True)
    axis.grid(which="minor", color="white", linewidth=1.5)
    axis.tick_params(which="minor", bottom=False, left=False)

    colorbar = fig.colorbar(image, ax=axes.ravel().tolist(), shrink=0.82, pad=0.02)
    colorbar.set_label("Valor de estado  $V\pi(s)$ ")
    fig.suptitle(
        "Evaluación de políticas – valores y acciones greedy inducidas",
        fontsize=17,
        fontweight="bold",
    )
    plt.show()

```

Evaluación de políticas — valores y acciones greedy inducidas



7. Comprobaciones y conclusión

Además de verificar las probabilidades del MDP, se comprueba que las tres evaluaciones cumplen el umbral solicitado y que la mejora greedy no reduce el valor del estado inicial frente a la política determinista de la que fue derivada.

```
In [7]: for policy_name, result in results.items():
        assert result["deltas"][-1] < THETA

        greedy_improvement = (
            greedy_values[initial_state] - fixed_values[initial_state]
        )
        assert greedy_improvement >= -1e-8

        best_policy_name = summary["V(base, lleva paquete)"].idxmax()
        display(
            Markdown(
                f"""Resultado principal.** Desde el estado inicial `{initial_state}`, "
                f"la política greedy mejora el valor de la determinista fija en "
                f"{greedy_improvement:.4f} unidades. Entre las tres políticas "
                f"evaluadas, el mayor valor inicial corresponde a **{best_policy_name}**."
                f"Todas convergieron con `Δ < {THETA}`."
            )
        )
```

Resultado principal. Desde el estado inicial (2, 2, 1), la política greedy mejora el valor de la determinista fija en **25.0398** unidades. Entre las tres políticas evaluadas, el mayor valor inicial corresponde a **Greedy sobre V determinista**. Todas convergieron con $\Delta < 1e-08$.

8. Análisis y conclusión final

Los resultados muestran una diferencia clara entre las tres políticas. La **política uniforme aleatoria** obtuvo un valor de **-24.2360** desde el estado inicial **(2, 2, 1)** y necesitó **337 iteraciones** para converger. El valor negativo se debe a que selecciona movimientos y esperas sin considerar el destino: acumula costos de tiempo y batería, tarda más en entregar y descuenta fuertemente las recompensas futuras.

La **política determinista fija** mejoró el valor inicial hasta **13.4983** y convergió en **50 iteraciones**. Aunque su recorrido serpenteante garantiza que eventualmente llegue al destino, no siempre utiliza la ruta más corta. Esto explica que sea mucho mejor que actuar al azar, pero todavía deje espacio para una mejora.

La **política greedy derivada de los valores de la determinista** alcanzó el mejor resultado: **38.5382** desde la base y convergió en **29 iteraciones**. Su mejora fue de **25.0398** unidades respecto de la política determinista. Los mapas confirman este comportamiento: durante la entrega, sus flechas apuntan por rutas directas hacia el destino y, después de entregar, conducen al dron hacia la base central.

El número de iteraciones no representa por sí solo la calidad de una política; indica cuánto tarda su tabla de valores en estabilizarse con el umbral $\theta = 1e-8$. En este caso, la política aleatoria converge más lentamente porque puede permanecer recorriendo el grid durante muchos pasos, mientras que las políticas deterministas producen trayectorias más predecibles.

Conclusión. La evaluación iterativa de políticas permitió comparar cuantitativamente estrategias distintas sin utilizar librerías de RL. La mejora greedy produjo la mejor de las tres políticas evaluadas, ya que maximiza el retorno esperado y reduce los costos acumulados antes de recibir las recompensas de entrega y regreso. El modelo cumple el objetivo del GridWorld 5×5, aunque sigue siendo una simplificación: utiliza un destino fijo, supone batería suficiente y no incorpora obstáculos, colisiones ni cambios dinámicos del viento. Esos elementos podrían añadirse en una extensión futura aumentando el espacio de estados y ajustando la función de transición.

Task 4 (Entrega Final)

Dictamen técnico dirigido a la Gerencia de Logística

Asunto: evaluación de políticas para el sistema de entrega urbana con drones.

Propósito: traducir los resultados del GridWorld 5×5 en una recomendación de negocio, cuantificar las diferencias entre políticas, revisar la sensibilidad al factor de descuento y

determinar si el modelo es una base suficiente para un sistema real de aprendizaje por refuerzo.

Para responder exactamente la pregunta de esta entrega se construye una política adicional: la **greedy derivada de los valores de la política uniforme aleatoria**. Esta se compara con la política uniforme y con la política determinista diseñada manualmente. La greedy de la Task 3, derivada de la determinista, se conserva como resultado previo.

1. Comparación cuantitativa de las políticas

La tabla resume el desempeño en el estado inicial —el dron está en la base y lleva el paquete— y en los 49 estados no terminales. Un valor más alto representa mayor retorno esperado después de considerar tiempo, batería, entrega, regreso y descuento.

```
In [8]: # Política solicitada para el dictamen: greedy derivada de V de la uniforme.
greedy_from_random_policy = greedy_policy_from_values(
    mdp, uniform_values, gamma=GAMMA
)
(
    greedy_from_random_values,
    greedy_from_random_iterations,
    greedy_from_random_deltas,
) = evaluate_policy_iteratively(
    mdp,
    greedy_from_random_policy,
    gamma=GAMMA,
    theta=THETA,
)

task4_results = OrderedDict(
    {
        "Uniforme aleatoria": {
            "policy": uniform_policy,
            "values": uniform_values,
            "iterations": uniform_iterations,
        },
        "Determinista manual": {
            "policy": fixed_policy,
            "values": fixed_values,
            "iterations": fixed_iterations,
        },
        "Greedy desde uniforme": {
            "policy": greedy_from_random_policy,
            "values": greedy_from_random_values,
            "iterations": greedy_from_random_iterations,
        },
    }
)

comparison_rows = []
for policy_name, result in task4_results.items():
    state_values = np.array(
```

```

        [result["values"][state] for state in mdp.nonterminal_states]
    )
    comparison_rows.append(
        {
            "Política": policy_name,
            "V inicial": result["values"][initial_state],
            "Promedio V": state_values.mean(),
            "Mínimo V": state_values.min(),
            "Máximo V": state_values.max(),
            "Iteraciones": result["iterations"],
        }
    )

task4_comparison = pd.DataFrame(comparison_rows).set_index("Política")
display(
    task4_comparison.style.format(
        {
            "V inicial": "{:.4f}",
            "Promedio V": "{:.4f}",
            "Mínimo V": "{:.4f}",
            "Máximo V": "{:.4f}",
        }
    )
)

# Estados con mayor separación entre las tres políticas.
state_spread_rows = []
for state in mdp.nonterminal_states:
    policy_values = {
        name: result["values"][state]
        for name, result in task4_results.items()
    }
    row, col, package = state
    state_spread_rows.append(
        {
            "Estado": str(state),
            "Fase": "entrega" if package == 1 else "retorno",
            "V uniforme": policy_values["Uniforme aleatoria"],
            "V determinista": policy_values["Determinista manual"],
            "V greedy": policy_values["Greedy desde uniforme"],
            "Rango max-min": (
                max(policy_values.values()) - min(policy_values.values())
            ),
        }
    )

largest_state_spreads = (
    pd.DataFrame(state_spread_rows)
    .sort_values("Rango max-min", ascending=False)
    .head(10)
    .reset_index(drop=True)
)
display(Markdown("### Estados donde las políticas difieren más"))
display(
    largest_state_spreads.style.format(
        {

```

```

        "V uniforme": "{:.3f}",
        "V determinista": "{:.3f}",
        "V greedy": "{:.3f}",
        "Rango max-min": "{:.3f}",
    }
)
)

greedy_vs_manual_gaps = {
    state: greedy_from_random_values[state] - fixed_values[state]
    for state in mdp.nonterminal_states
}
greedy_better_states = sum(
    gap > 1e-8 for gap in greedy_vs_manual_gaps.values()
)
greedy_equal_states = sum(
    abs(gap) <= 1e-8 for gap in greedy_vs_manual_gaps.values()
)
manual_better_states = sum(
    gap < -1e-8 for gap in greedy_vs_manual_gaps.values()
)
largest_gap_state = max(greedy_vs_manual_gaps, key=greedy_vs_manual_gaps.get)
largest_gap = greedy_vs_manual_gaps[largest_gap_state]
initial_greedy_advantage = (
    greedy_from_random_values[initial_state] - fixed_values[initial_state]
)

print(
    f"Greedy supera a la manual en {greedy_better_states} estados "
    f"y empata en {greedy_equal_states}; la manual es mejor en "
    f"{manual_better_states}."
)
print(
    f"Mayor ventaja greedy/manual: {largest_gap:.4f} "
    f"en {largest_gap_state}."
)

```

	V inicial	Promedio V	Mínimo V	Máximo V	Iteraciones
Política					
Uniforme aleatoria	-24.2360	-13.2537	-25.8600	5.9712	337
Determinista manual	13.4983	24.7887	-8.0071	56.0324	50
Greedy desde uniforme	38.5382	29.8853	-20.0000	56.0324	361

Estados donde las políticas difieren más

	Estado	Fase	V uniforme	V determinista	V greedy	Rango max-min
0	(0, 2, 1)	entrega	-21.161	46.789	46.789	67.950
1	(2, 4, 1)	entrega	-21.161	18.735	46.789	67.950
2	(1, 4, 1)	entrega	-15.565	21.585	51.279	66.844
3	(0, 3, 1)	entrega	-15.565	51.279	51.279	66.844
4	(1, 3, 1)	entrega	-19.772	24.601	46.789	66.561
5	(3, 4, 1)	entrega	-23.660	-0.779	42.546	66.206
6	(0, 1, 1)	entrega	-23.660	42.546	42.546	66.206
7	(2, 3, 1)	entrega	-22.776	16.042	42.546	65.322
8	(1, 2, 1)	entrega	-22.776	27.794	42.546	65.322
9	(4, 4, 1)	entrega	-24.515	-2.393	38.538	63.053

Greedy supera a la manual en 13 estados y empata en 30; la manual es mejor en 6. Mayor ventaja greedy/manual: 43.3252 en (3, 4, 1).

```
In [9]: # Comparación visual: estado inicial y brecha greedy - determinista por fase.
fig, axes = plt.subplots(1, 3, figsize=(17, 5), constrained_layout=True)

policy_names = list(task4_results)
initial_values = [
    task4_results[name]["values"][initial_state] for name in policy_names
]
bar_colors = ["#94a3b8", "#2563eb", "#16a34a"]
axes[0].bar(policy_names, initial_values, color=bar_colors)
axes[0].axhline(0, color="#334155", linewidth=1)
axes[0].set_title("Valor desde la base con paquete")
axes[0].set_ylabel("V $\pi(2, 2, 1)$ ")
axes[0].tick_params(axis="x", rotation=18)
for index, value in enumerate(initial_values):
    axes[0].text(
        index,
        value + (1.0 if value >= 0 else -2.0),
        f"{value:.2f}",
        ha="center",
        va="bottom" if value >= 0 else "top",
        fontweight="bold",
    )

gap_values = {
    state: greedy_from_random_values[state] - fixed_values[state]
    for state in mdp.nonterminal_states
}
gap_limit = max(abs(value) for value in gap_values.values())

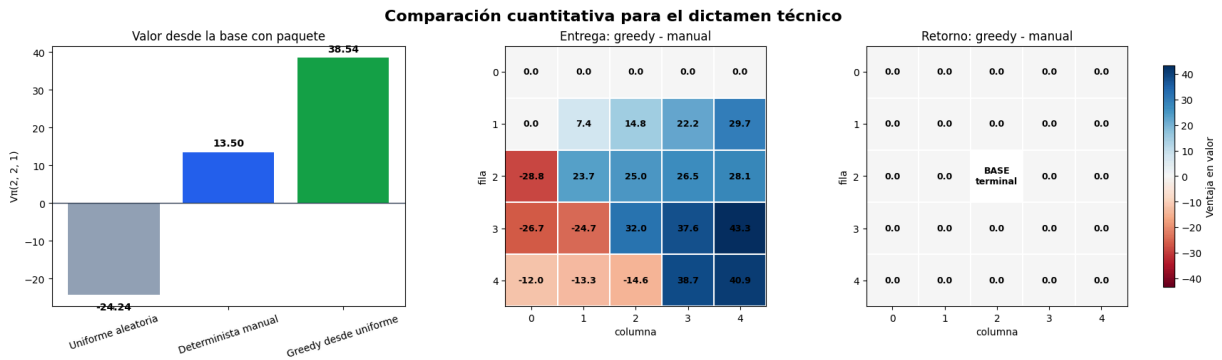
for axis, package, phase_title in zip(
    axes[1:],
    (1, 0),
    ("Entrega: greedy - manual", "Retorno: greedy - manual"),
)
```

```

):
    grid = value_grid(gap_values, package).to_numpy()
    image = axis.imshow(
        np.ma.masked_invalid(grid),
        cmap="RdBu",
        vmin=-gap_limit,
        vmax=gap_limit,
    )
    for row in range(mdp.grid_size):
        for col in range(mdp.grid_size):
            if np.isfinite(grid[row, col]):
                axis.text(
                    col,
                    row,
                    f"{grid[row, col]:.1f}",
                    ha="center",
                    va="center",
                    fontsize=9,
                    fontweight="semibold",
                )
            else:
                axis.text(
                    col,
                    row,
                    "BASE\nterminal",
                    ha="center",
                    va="center",
                    fontsize=9,
                    fontweight="bold",
                )
    axis.set_title(phase_title)
    axis.set_xlabel("columna")
    axis.set_ylabel("fila")
    axis.set_xticks(range(mdp.grid_size))
    axis.set_yticks(range(mdp.grid_size))
    axis.set_xticks(np.arange(-0.5, mdp.grid_size, 1), minor=True)
    axis.set_yticks(np.arange(-0.5, mdp.grid_size, 1), minor=True)
    axis.grid(which="minor", color="white", linewidth=1.5)
    axis.tick_params(which="minor", bottom=False, left=False)

colorbar = fig.colorbar(image, ax=axes[1:].tolist(), shrink=0.85)
colorbar.set_label("Ventaja en valor")
fig.suptitle(
    "Comparación cuantitativa para el dictamen técnico",
    fontsize=16,
    fontweight="bold",
)
plt.show()

```

2. Sensibilidad al factor de descuento γ

Se repite el procedimiento con $\gamma = 0.80$ y $\gamma = 0.99$, además del valor 0.95 propuesto en la Tarea 1. Para cada γ se vuelven a evaluar las políticas uniforme y determinista, se deriva una nueva política greedy desde los valores de la uniforme y se evalúa esa política.

También se cuenta cuántos estados cambian de acción greedy respecto a $\gamma = 0.95$. Esto permite distinguir un cambio puramente numérico en los valores de un cambio real en las decisiones.

```
In [10]: GAMMA_VALUES = (0.80, 0.95, 0.99)
sensitivity_rows = []
greedy_policies_by_gamma = {}
sensitivity_values = {}

for gamma_value in GAMMA_VALUES:
    if np.isclose(gamma_value, GAMMA):
        uniform_v = uniform_values
        uniform_n = uniform_iterations
        fixed_v = fixed_values
        fixed_n = fixed_iterations
        greedy_v = greedy_from_random_values
        greedy_n = greedy_from_random_iterations
        gamma_greedy_policy = greedy_from_random_policy
    else:
        uniform_v, uniform_n, _ = evaluate_policy_iteratively(
            mdp, uniform_policy, gamma=gamma_value, theta=THETA
        )
        fixed_v, fixed_n, _ = evaluate_policy_iteratively(
            mdp, fixed_policy, gamma=gamma_value, theta=THETA
        )
        gamma_greedy_policy = greedy_policy_from_values(
            mdp, uniform_v, gamma=gamma_value
        )
        greedy_v, greedy_n, _ = evaluate_policy_iteratively(
            mdp,
            gamma_greedy_policy,
            gamma=gamma_value,
            theta=THETA,
        )
```

```

greedy_policies_by_gamma[gamma_value] = gamma_greedy_policy
sensitivity_values[gamma_value] = {
    "uniforme": uniform_v,
    "determinista": fixed_v,
    "greedy": greedy_v,
}
sensitivity_rows.append(
    {
        "γ": gamma_value,
        "V inicial uniforme": uniform_v[initial_state],
        "V inicial determinista": fixed_v[initial_state],
        "V inicial greedy": greedy_v[initial_state],
        "Iter. uniforme": uniform_n,
        "Iter. determinista": fixed_n,
        "Iter. greedy": greedy_n,
    }
)

def policy_action(policy, state):
    return max(policy[state], key=policy[state].get)

baseline_greedy = greedy_policies_by_gamma[GAMMA]
for row in sensitivity_rows:
    gamma_value = row["γ"]
    row["Estados con acción distinta vs γ=0.95"] = sum(
        policy_action(greedy_policies_by_gamma[gamma_value], state)
        != policy_action(baseline_greedy, state)
        for state in mdp.nonterminal_states
    )

sensitivity_table = pd.DataFrame(sensitivity_rows).set_index("γ")
display(
    sensitivity_table.style.format(
        {
            "V inicial uniforme": "{:.4f}",
            "V inicial determinista": "{:.4f}",
            "V inicial greedy": "{:.4f}",
        }
    )
)

changed_policy_rows = []
for gamma_value in GAMMA_VALUES:
    if np.isclose(gamma_value, GAMMA):
        continue
    for state in mdp.nonterminal_states:
        baseline_action = policy_action(baseline_greedy, state)
        gamma_action = policy_action(
            greedy_policies_by_gamma[gamma_value], state
        )
        if gamma_action != baseline_action:
            changed_policy_rows.append(
                {

```

```

        "γ": gamma_value,
        "Estado": str(state),
        "Acción con γ=0.95": baseline_action,
        "Nueva acción": gamma_action,
    }
)

if changed_policy_rows:
    display(Markdown("### Estados donde cambia la política greedy"))
    display(pd.DataFrame(changed_policy_rows))
else:
    display(
        Markdown(
            "***Robustez de decisiones:** ninguna acción greedy cambió "
            "en los 49 estados para los valores de γ evaluados."
        )
    )
)

```

	V inicial uniforme	V inicial determinista	V inicial greedy	Iter. uniforme	Iter. determinista	Iter. greedy	Estados con acción distinta vs γ=0.95
γ							
0.800000	-6.6979	-4.9312	-5.0000	84	45	84	9
0.950000	-24.2360	13.4983	38.5382	337	50	361	0
0.990000	-89.9270	35.2105	52.5080	1271	51	30	6

Estados donde cambia la política greedy

	γ	Estado	Acción con $\gamma=0.95$	Nueva acción
0	0.80	(0, 0, 1)	ESTE	ESPERAR
1	0.80	(1, 0, 1)	ESTE	ESPERAR
2	0.80	(1, 1, 1)	ESTE	ESPERAR
3	0.80	(2, 1, 1)	ESTE	ESPERAR
4	0.80	(2, 2, 1)	NORTE	ESPERAR
5	0.80	(3, 2, 1)	NORTE	ESPERAR
6	0.80	(3, 3, 1)	NORTE	ESPERAR
7	0.80	(4, 3, 1)	NORTE	ESPERAR
8	0.80	(4, 4, 1)	NORTE	ESPERAR
9	0.99	(2, 0, 1)	ESPERAR	NORTE
10	0.99	(3, 0, 1)	ESPERAR	NORTE
11	0.99	(3, 1, 1)	ESPERAR	NORTE
12	0.99	(4, 0, 1)	ESPERAR	NORTE
13	0.99	(4, 1, 1)	ESPERAR	ESTE
14	0.99	(4, 2, 1)	ESPERAR	ESTE

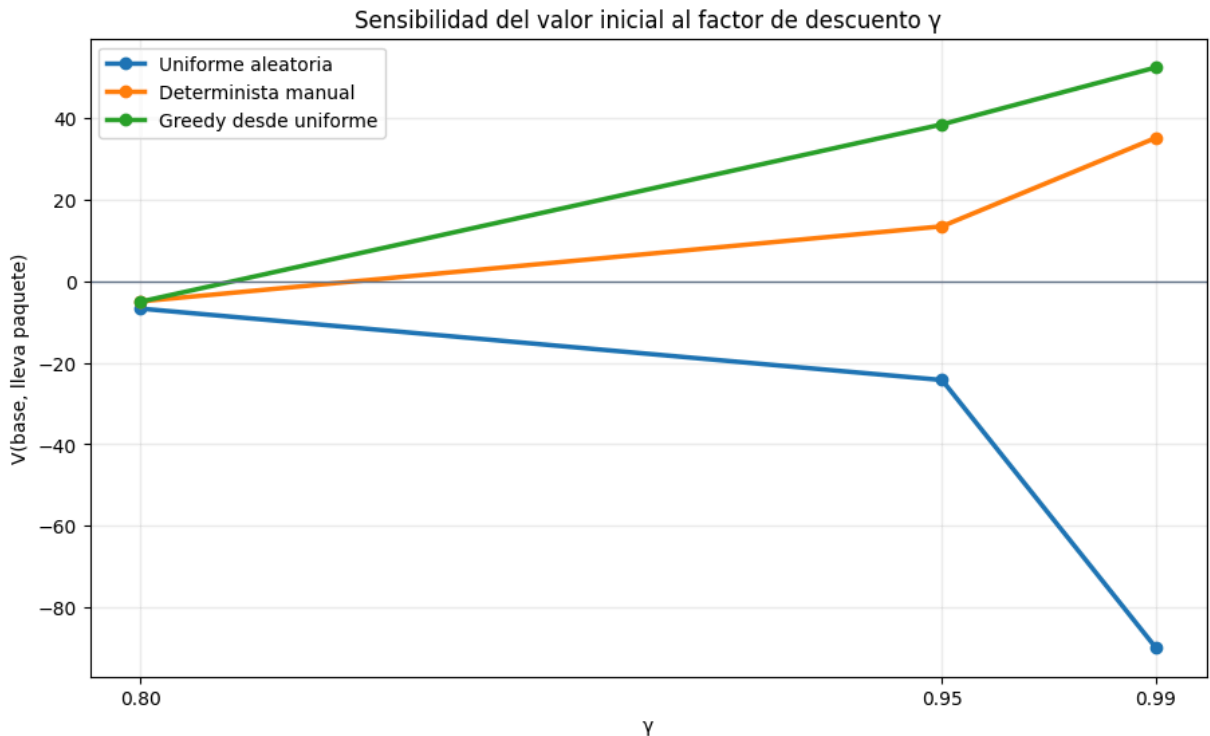
```
In [11]: fig, axis = plt.subplots(figsize=(9, 5.5), constrained_layout=True)

axis.plot(
    sensitivity_table.index,
    sensitivity_table["V inicial uniforme"],
    marker="o",
    linewidth=2.5,
    label="Uniforme aleatoria",
)
axis.plot(
    sensitivity_table.index,
    sensitivity_table["V inicial determinista"],
    marker="o",
    linewidth=2.5,
    label="Determinista manual",
)
axis.plot(
    sensitivity_table.index,
    sensitivity_table["V inicial greedy"],
    marker="o",
    linewidth=2.5,
    label="Greedy desde uniforme",
)
axis.axhline(0, color="#64748b", linewidth=1)
axis.set_title("Sensibilidad del valor inicial al factor de descuento  $\gamma$ ")
axis.set_xlabel(" $\gamma$ ")
```

```

axis.set_ylabel("V(base, lleva paquete)")
axis.set_xticks(GAMMA_VALUES)
axis.grid(alpha=0.25)
axis.legend()
plt.show()

```



```

In [12]: winning_policy = task4_comparison["V inicial"].idxmax()
top_spread = largest_state_spreads.iloc[0]

gamma_low = sensitivity_table.loc[0.80]
gamma_base = sensitivity_table.loc[0.95]
gamma_high = sensitivity_table.loc[0.99]
changes_low = int(
    gamma_low["Estados con acción distinta vs  $\gamma=0.95$ "]
)
changes_high = int(
    gamma_high["Estados con acción distinta vs  $\gamma=0.95$ "]
)

comparison_verdict = (
    f"### Dictamen sobre desempeño\n\n"
    f"La política con mayor valor desde el estado inicial es "
    f"{winning_policy}, con "
    f"V={task4_comparison.loc[winning_policy, 'V inicial']:.4f}`. "
    f"La greedy derivada de la aleatoria supera a la política manual por "
    f"{initial_greedy_advantage:.4f} unidades desde la base; además, "
    f"la supera en {greedy_better_states} de 49 estados y empata en "
    f"{greedy_equal_states}. La política manual conserva ventaja en "
    f"{manual_better_states} estados, por lo que la superioridad greedy "
    f"no es universal aunque sí es clara desde la base. La separación máxima entre "
    f"políticas ocurre en el estado `{top_spread['Estado']}`, durante la "
    f"fase de {top_spread['Fase']}, con un rango de "
    f"{top_spread['Rango max-min']:.4f} unidades.\n\n"

```

```
f"### Dictamen sobre sensibilidad\n\n"
f"Al pasar de `γ=0.80` a `γ=0.95` y `γ=0.99`, el valor inicial de la "
f"greedy cambia de  $\gamma_{low}$ ['V inicial greedy']:.4f** a "
f" $\gamma_{base}$ ['V inicial greedy']:.4f** y "
f" $\gamma_{high}$ ['V inicial greedy']:.4f**, respectivamente. "
f"Los cambios de acción frente a `γ=0.95` son  $\Delta$  para "
f"`γ=0.80` y  $\Delta$  para `γ=0.99`. Un γ alto da más peso "
f"a las recompensas futuras de entrega y retorno, pero también conserva "
f"por más tiempo los costos de una política que deambula. Un γ menor "
f"descuenta tan rápido las recompensas lejanas que la greedy derivada de "
f"la aleatoria prefiere ESPERAR en varios estados; con `γ=0.80`, "
f"esto incluye el estado inicial y produce `V=-5.0000`, ligeramente peor "
f"que la manual ( $V=\gamma_{low}$ ['V inicial determinista']:.4f` ). "
f"Por tanto, γ modifica tanto la magnitud de los valores como decisiones "
f"operacionales importantes."
)
display(Markdown(comparison_verdict))
```

Dictamen sobre desempeño

La política con mayor valor desde el estado inicial es **Greedy desde uniforme**, con $V=38.5382$. La greedy derivada de la aleatoria supera a la política manual por **25.0398** unidades desde la base; además, la supera en **13 de 49 estados** y empata en **30**. La política manual conserva ventaja en **6 estados**, por lo que la superioridad greedy no es universal aunque sí es clara desde la base. La separación máxima entre las tres políticas ocurre en el estado $(0, 2, 1)$, durante la fase de **entrega**, con un rango de **67.9499** unidades.

Dictamen sobre sensibilidad

Al pasar de $\gamma=0.80$ a $\gamma=0.95$ y $\gamma=0.99$, el valor inicial de la greedy cambia de **-5.0000** a **38.5382** y **52.5080**, respectivamente. Los cambios de acción frente a $\gamma=0.95$ son **9** para $\gamma=0.80$ y **6** para $\gamma=0.99$. Un γ alto da más peso a las recompensas futuras de entrega y retorno, pero también conserva por más tiempo los costos de una política que deambula. Un γ menor descuenta tan rápido las recompensas lejanas que la greedy derivada de la aleatoria prefiere **ESPERAR** en varios estados; con $\gamma=0.80$, esto incluye el estado inicial y produce $V=-5.0000$, ligeramente peor que la manual ($V=-4.9312$). Por tanto, γ modifica tanto la magnitud de los valores como decisiones operacionales importantes.

3. Limitaciones frente a la logística urbana real

El MDP es útil como experimento controlado, pero omite factores que condicionan una operación real:

- **Estado físico incompleto:** la batería se representa como costo, no como nivel, temperatura, degradación o autonomía restante. Tampoco se modelan carga útil, salud de motores ni mantenimiento.

- **Entorno demasiado estático:** no hay edificios, cables, zonas de exclusión, tráfico aéreo, peatones, aves ni otros drones. El viento solo causa permanencia en la celda y no tiene dirección, intensidad ni dependencia temporal.
- **Observación perfecta:** se supone que el dron conoce su celda y la fase del paquete sin error. En la ciudad real, GPS, cámara y sensores producen observaciones parciales y ruidosas; sería más apropiado un POMDP.
- **Una misión y un destino fijo:** no se consideran múltiples pedidos, prioridades, ventanas horarias, capacidad de flota, recarga, asignación de vehículos ni replanificación.
- **Recompensas hipotéticas:** los valores `-1.5` , `+30` y `+40` no están calibrados con costos financieros, nivel de servicio, consumo medido o riesgo real.
- **Seguridad y regulación ausentes:** no hay probabilidad explícita de accidente, restricciones aeronáuticas, privacidad, redundancia, aterrizaje de emergencia ni supervisión humana.
- **Grid discreto y pequeño:** una cuadrícula 5×5 no captura distancias, altitud, velocidad, aceleración, trayectorias continuas ni el tamaño de una ciudad.

Para hacerlo operacionalmente útil se necesitaría ampliar el estado con batería y condiciones ambientales, incorporar mapas y zonas prohibidas, usar transiciones estimadas con telemetría, modelar observabilidad parcial, incluir restricciones de seguridad y simular múltiples drones y pedidos. La función de recompensa debe calibrarse con indicadores empresariales y límites regulatorios.

4. Recomendación concreta a la gerencia

Recomendación: no utilizar todavía este MDP como controlador de una flota real. Sí es una **base conceptual suficiente para validar algoritmos, interfaces y supuestos en un simulador**, pero no una base operacional suficiente para autorizar vuelos o invertir directamente en una implementación completa de RL.

Antes de proceder se recomiendan los siguientes pasos:

1. **Recolectar y gobernar datos reales** de rutas, consumo, viento, entregas fallidas, incidentes y tiempos de servicio.
2. **Construir un simulador o gemelo digital** con mapas, clima, zonas prohibidas, batería explícita, pedidos y múltiples drones.
3. **Calibrar recompensas y transiciones** con métricas financieras, seguridad, puntualidad y restricciones legales acordadas con operaciones.
4. **Definir una arquitectura segura**, con acciones restringidas, verificación formal de límites, reglas de emergencia y posibilidad de intervención humana.
5. **Comparar RL contra baselines operativos**, como rutas más cortas, planificación matemática y control basado en reglas. RL solo debe avanzar si demuestra una mejora consistente.

6. **Realizar pruebas escalonadas:** simulación, evaluación offline, hardware-in-the-loop, zona controlada y finalmente un piloto pequeño con monitoreo.
7. **Establecer criterios de aprobación** antes del piloto: tasa máxima de incidentes, autonomía mínima, puntualidad, ahorro esperado y capacidad de auditoría.

Conclusión ejecutiva: continuar con una fase de investigación y simulación, no con un despliegue productivo. La evidencia del GridWorld muestra que la mejora de políticas funciona y que la estrategia greedy ofrece el mejor valor desde la base con $\gamma = 0.95$; sin embargo, no domina a la política manual en todos los estados y puede aprender a esperar cuando el descuento hace poco atractiva la misión. Esta sensibilidad refuerza que el salto entre el entorno académico y la logística urbana real sigue siendo demasiado grande para justificar operación autónoma sin calibración, datos, simulación avanzada y controles de seguridad.